ShopEZ - MERN Stack E-Commerce Application

ShopEZ is a polished MERN stack e-commerce application with a clean, modern design inspired by popular online marketplaces. It uses React (Vite) on the frontend and Node/Express/MongoDB on the backend, complete with user authentication, cart management, checkout flow, and an admin dashboard.

■ Features

■ User Side

Home Page: Interactive landing page with a hero banner, featured category cards, and trending items.

Product Catalog: A comprehensive products page with dynamic search, category sidebar filters, and sorting capabilities.

Product Details: Dedicated page for each product showing image, description, rating, price, and stock status.

Authentication: Complete JWT-based registration and login system with client-side route guards.

Shopping Cart: Add, update quantities, or remove products in real-time.

Checkout Flow: Simple checkout form for shipping address and payment method selection.

Order Tracking: Profile page detailing order history and current status.

■ Admin Side

Default Credentials:

Email: `admin@shopez.com`

Password: `admin123`

Admin Dashboard: Overview of key statistics:

Total Registered Users

Total Available Products

Total Placed Orders

Overall Revenue (dynamically calculated)

Product Management: Create, edit, and delete products from the interface.

Order Management: View all customer orders and update their delivery status (Pending, Shipped, Delivered).

■■ Tech Stack

Frontend: React, Vite, React Router DOM, Axios, Context API, Vanilla CSS.

Backend: Node.js, Express.js, MongoDB + Mongoose.

Security & Auth: JSON Web Tokens (JWT), BcryptJS.

Assets & Styling: Lucide React icons, premium typography (Plus Jakarta Sans & Outfit), clean and responsive styling.

■ Folder Structure

ShopEZ/

■

■■■ client/

■ ■■■ src/

■ ■ ■■■ assets/ # Styles, images, and fonts

■ ■ ■■■ components/ # Header, Footer, Sidebar, and Product Card components

■ ■ ■■■ context/ # AuthContext and CartContext

■ ■ ■■■ pages/ # Home, Products, Profile, Cart, Checkout, Dashboard, etc.

■ ■ ■■■ services/ # Axios API service client

■ ■ ■■■ App.jsx # App layout & routing table

■ ■ ■■■ main.jsx # Entrypoint

■ ■■■ package.json

■

■■■ server/

■ ■■■ config/ # DB connection & database seeder

■ ■■■ controllers/ # Route handler controllers (Auth, Products, Cart, Orders, Admin)

■ ■■■ middleware/ # JWT auth & role validation middleware

■ ■■■ models/ # Mongoose schemas (User, Product, Cart, Order)

■ ■■■ routes/ # REST API routes mapping

■ ■■■ index.js # Express server entry point

■ ■■■ package.json

■

■■■ README.md

■■■ .env.example

■ API Routes Reference

Authentication

`POST /api/auth/register` - Create user account
`POST /api/auth/login` - User sign-in
`POST /api/admin/login` - Admin credentials login

Products

`GET /api/products` - Fetch all products (supports `search`, `category`, `gender`, `sort`)
`GET /api/products/:id` - Fetch single product details
`POST /api/products` - Add new product (Admin only)
`PUT /api/products/:id` - Edit product (Admin only)
`DELETE /api/products/:id` - Delete product (Admin only)

Cart

`GET /api/cart` - Retrieve user cart
`POST /api/cart/add` - Add item to cart
`DELETE /api/cart/remove` - Remove item from cart

Orders

`POST /api/orders` - Place a new order
`GET /api/orders/user` - Fetch current user's order history
`GET /api/orders/all` - Fetch all orders (Admin only)
`PUT /api/orders/status` - Update order status (Admin only)

Admin Panel

`GET /api/admin/dashboard` - Get high-level stats (Admin only)

■ Running Locally on Windows

1. Prerequisite Setup

If you want persistent database storage, ensure a local instance of **MongoDB** is running:

Connection URL: `mongodb://localhost:27017/shopez`

[!NOTE]

***Automatic Fallback Mode**: If MongoDB is not running locally, ShopEZ will automatically initialize in a **Mock In-Memory Database Mode**. All actions (registration, log in, cart management, checkout, order placing, admin panel edit/delete operations) will work perfectly in-memory. Data will reset if you restart the server.*

2. Configure Environment Variables

Copy .env.example in the server or root folder to .env:

cp .env.example .env

3. Start the Backend Server

cd server

npm install

node index.js

The server will start on port 5000 and seed the initial admin account and 50 premium products.

4. Start the Frontend Dev Server

Open a new terminal window:

cd client

npm install

npm run dev

Open your browser to the URL outputted by Vite (typically http://localhost:5173).